VeriSight Architectural & Codebase Report

AI-Powered Autonomous Multi-Agent Framework for ASIC Simulation Debugging & Root Cause Analysis

---

1. Executive Summary

VeriSight is a production-grade, autonomous multi-agent software framework designed for root-cause analysis of ASIC/FPGA simulation failures. Functioning as a virtual senior verification engineer, VeriSight systematically traces issues from initial failure symptoms in simulation logs down to exact hardware register level or testbench bug causes with rigorous evidence chains.

The system seamlessly combines deterministic SystemVerilog/UVM parsers, gate-level X-propagation analyzers, a persistent RAG (Retrieval-Augmented Generation) knowledge base built on ChromaDB, and state-of-the-art LLM reasoning (Google Gemini & Anthropic Claude) to achieve highly accurate diagnostic reports and automated code fixes.

2. System Architecture & Autonomous Pipeline

VeriSight operates as an orchestrated multi-agent pipeline composed of five distinct specialized AI Agents working in sequence, backed by high-performance parsers, hardware tool adapters, and vector memory:

Stage	Agent Name	Primary Role	Key Outputs
Agent 1	**Knowledge Extraction**	Parses Spec, RTL, TB, Log, Coverage, and VCD waveforms; indexes data & queries RAG	knowledge.json, spec.json, rtl.json, tb.json, log.json
Agent 2	**Root Cause Classifier**	Heuristic pre-analysis & multi-hypothesis LLM reasoning (RTL Bug, TB Bug, Spec Bug, Unknown)	summary.json
Agent 3	**RTL Root Cause Analyzer**	Runs 7 sub-module analyzers (X-Trace, Functional, CDC, Lint, Structural, Protocol, Misc)	analysis/*.json
Agent 4	**Report Generator**	Synthesizes data into human & machine readable markdown, HTML, text, and JSON reports	error.json, report.md, report.html, summary.txt
Agent 5	**Automated Fix Generator**	Generates exact code diff patches when confidence threshold (default >=70%) is met	fix.json, applied patches

3. Complete Directory & File Breakdown

3.1 Core CLI & Orchestrator

- **main.py**: CLI entry point. Parses flags (--spec, --rtl, --tb, --log, --simulate, --xtrace, --no-rag, --provider, --model), initializes config & logging, and invokes VeriSightOrchestrator.
- **verisight/config.py**: Global configuration management using Pydantic Settings. Controls LLM providers (Gemini/Anthropic), model names, ChromaDB paths, log levels, simulator choices, and tool binary paths.

• **verisight/orchestrator.py**: Central orchestrator (VeriSightOrchestrator). Manages execution flow through Agents 1-5, passes structured knowledge objects, handles error fallbacks, and persists debug sessions in RAG.

3.2 AI Agents (verisight/agents/)

• **verisight/agents/base_agent.py**: Abstract base class for all AI agents. Encapsulates LLM client initialization (Gemini GenAI & Anthropic SDK), structured JSON response parsing, prompt generation, and retry logic.

• **verisight/agents/agent1_knowledge.py**: Knowledge Extraction Agent. Orchestrates deterministic parsers (Spec, RTL, UVM, Log, Coverage, VCD), aggregates results into KnowledgeBase, and queries ChromaDB for historical vector matches.

• **verisight/agents/agent2_classifier.py**: Root Cause Classifier Agent. Runs deterministic pre-analysis heuristics and multi-hypothesis LLM reasoning to classify bugs into RTL Bug, TB Bug, Spec Bug, or Unknown.

• **verisight/agents/agent3_rtl_analyzer.py**: RTL Analyzer Agent. Triggers 7 sub-modules: X-Trace, Functional, CDC, Lint, Structural, Protocol, Misc, saving fine-grained findings into analysis/*.json.

• **verisight/agents/agent4_reporter.py**: Report Generator Agent. Formats findings into Markdown (report.md), HTML (report.html), text executive summary (summary.txt), and machine-readable error.json using Jinja2 templates.

• **verisight/agents/agent5_fix_generator.py**: Automated Fix Generator Agent. Evaluates confidence thresholds, constructs code diff recommendations, and produces fix.json.

3.3 Deterministic Parsers (verisight/parsers/)

• **verisight/parsers/spec_parser.py**: Parses Markdown design specs. Extracts functional requirements, register definitions, bitfields, and FSM state requirements into Pydantic models.

• **verisight/parsers/rtl_parser.py**: SystemVerilog/Verilog AST & regex parser. Identifies modules, ports, clocking domains, reset signals, processes (always, always_ff, always_comb), and lint warnings.

• **verisight/parsers/uvm_parser.py**: UVM testbench parser. Extracts drivers, monitors, scoreboards, virtual sequencers, transaction classes, objects, and component hierarchies from SystemVerilog files.

• **verisight/parsers/log_parser.py**: Simulation log analyzer. Extracts UVM_ERROR, UVM_FATAL, assertion failures, scoreboard mismatches, expected vs actual value pairs, and simulation timestamps.

• **verisight/parsers/coverage_parser.py**: Parses code and functional coverage reports (.rpt/.txt), extracting line, toggle, FSM, and covergroup percentages.

• **verisight/parsers/vcd_parser.py**: High-performance VCD waveform parser. Uses pywellen (Rust-backed) when available with a vcdvcd Python fallback to extract signal activity and X/Z transitions near failure timestamps.

3.4 Data Schemas (verisight/schemas/)

• **verisight/schemas/spec_schema.py**: Defines Requirement, RegisterDef, FSMDef, and SpecKnowledge Pydantic models.

• **verisight/schemas/rtl_schema.py**: Defines RTLModule, RTLPort, RTLProcess, RTLIssue, and RTLKnowledge models.

• **verisight/schemas/tb_schema.py**: Defines TBDriver, TBMonitor, TBScoreboard, TBSequence, and TBKnowledge models.

• **verisight/schemas/log_schema.py**: Defines LogError, AssertionFailure, ScoreboardMismatch, and LogKnowledge models.

• **verisight/schemas/vcd_schema.py**: Defines VCDData, SignalChange, and MismatchCorroboration waveform schemas.

• **verisight/schemas/knowledge_schema.py**: Defines master KnowledgeBase aggregating all input domain knowledge.

• **verisight/schemas/classification.py**: Defines ClassificationResult, confidence scores, bug categories, and evidence chains.

• **verisight/schemas/rtl_analysis.py**: Defines sub-module results for X-Trace, CDC, Lint, Protocol, and Structural checks.

• **verisight/schemas/report_schema.py**: Defines complete report output data structures.

• **verisight/schemas/fix_schema.py**: Defines CodeFix, diff blocks, and FixResult structures.

3.5 RAG Knowledge Subsystem (verisight/rag/)

• **verisight/rag/knowledge_base.py**: Manages ChromaDB vector store across 11 specialized collections (RTL bugs, TB bugs, Spec mismatches, X-prop patterns, CDC issues, etc.).

• **verisight/rag/embeddings.py**: Generates semantic text embeddings using SentenceTransformers with lightweight fallback generators.

• **verisight/rag/retriever.py**: Retrieves relevant past failure cases and bug fix patterns based on current error signatures.

3.6 Hardware Tool Adapters (verisight/tools/)

• **verisight/tools/sim_runner.py**: Internal simulation runner. Invokes Icarus Verilog (iverilog/vvp) or Verilator, injecting $dumpfile/$dumpvars for automatic VCD waveform generation.

• **verisight/tools/yosys_runner.py**: Invokes Yosys open-source synthesis suite to synthesize gate-level netlists (.v) from SystemVerilog RTL.

• **verisight/tools/xtracer_runner.py**: Wraps x-tracer tool to perform backward X-propagation tracing through synthesized gate-level netlists against VCD waveforms.

3.7 System Utilities (verisight/utils/)

• **verisight/utils/file_utils.py**: Robust file system utilities for reading/writing JSON, text, and markdown files safely.

• **verisight/utils/logger.py**: Configures Rich console logging with colored formatting and debug levels.

• **verisight/utils/sv_lexer.py**: SystemVerilog lexer for tokenizing Verilog/SystemVerilog source code.

3.8 Report Templates (templates/)

• **templates/report.md.j2**: Jinja2 template rendering comprehensive Markdown reports (report.md).

• **templates/report.html.j2**: Jinja2 template rendering styled HTML reports with interactive CSS styling (report.html).

• **templates/summary.txt.j2**: Jinja2 template rendering plain-text executive summaries (summary.txt).

3.9 Projects & Examples (projects/, examples/)

• **examples/rtl/alu.sv & examples/tb/alu_*.sv**: ALU example demonstration files containing a deliberate missing reset bug on result register.

• **projects/fifo/**: Synchronous FIFO project containing RTL (syn_fifo.sv), UVM testbench (25 components), specification (spec.md), and simulation log (sim.log).

3.10 Automated Test Suite (tests/)

• **tests/test_agents.py**: Unit tests for AI Agent prompt construction and classification logic.

• **tests/test_fix_generator.py**: Unit tests for Agent 5 fix thresholding and diff generation.

• **tests/test_llm_adapter.py**: Unit tests for LLM client integration (Gemini & Anthropic adapters).

• **tests/test_orchestrator.py**: Integration tests for pipeline orchestration and data flow.

- **tests/test_parsers.py**: Unit tests for RTL, Spec, UVM, Log, and VCD parsers.
- **tests/test_tools.py**: Unit tests for simulation, Yosys, and X-Tracer tool runners.

4. Summary & Verification Capabilities

VeriSight presents an end-to-end framework combining deterministic verification tools with intelligent multi-agent AI reasoning. By structuring debugging into discrete agent workflows (Extract → Classify → Analyze → Report → Fix), VeriSight enables automated triage, reduced debug turnaround time, and persistent historical knowledge retention for complex ASIC simulation failures.